

Gym-Util: Fitnessstudioauslastungsprognose mithilfe von Zeitreihenmodellen

Projektbericht

für die Prüfung zum
Bachelor of Science

des Studiengangs Wirtschaftsinformatik –

an der

Dualen Hochschule Baden-Württemberg

12. Juli 2026

Kurs

Ausbildungsfirma

Betreuer der Ausbildungsfirma

Wissenschaftlicher Betreuer

Helvetia Versicherungen,

n/a

n/a

Ehrenwörtliche Erklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit mit dem Thema

Gym-Util: Fitnessstudioauslastungsprognose mithilfe von Zeitreihenmodellen
selbstständig und ohne Benutzung anderer als der angegebenen Quellen und
Hilfsmittel angefertigt habe.

Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten und nicht veröffentlichten Schriften entnommen wurden, sind als solche kenntlich gemacht.

Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt

Die Arbeit ist in gleicher oder ähnlicher Form oder auszugsweise im Rahmen einer anderen Prüfung noch nicht vorgelegt worden.

██████████, 12. Juli 2026

11/11/2016

Hinweis zum Umfang der Arbeit

Der Textteil der vorliegenden Arbeit - beginnend mit der Einleitung bis ausschließlich Quellenverzeichnis - umfasst 23 Seiten.

Kurzfassung

Die Vorhersage der Fitnessstudioauslastung ist aufgrund von nichtlinearen Einflussfaktoren wie Wetter und Feiertagen hochkomplex. Diese Arbeit modelliert das Besucherverhalten prädiktiv durch datengetriebene Machine-Learning-Verfahren. Im Fokus der automatisierten Datenpipeline standen ein methodisch sauberes Feature-Engineering (u. a. autoregressive Lags) sowie die strikte Vermeidung von Data Leakage durch Walk-Forward-Validierung (TimeSeriesSplit).

Die Architektur teilt sich in ein „Quick-Modell“ für die grobe Tagesplanung sowie sechs „Detailed-Modelle“ für Kurzzeithorizonte (5–120 Min). Baumbasierte Ensemble-Verfahren (insb. XGBoost) erwiesen sich linearen Modellen als signifikant überlegen, da sie nichtlineare Interaktionseffekte abbilden können. Bei einem Prognosehorizont von 5 Minuten wurde ein Bestimmtheitsmaß (R^2) von 0,96 erreicht, bei zwei Stunden 0,68. Trotz der aktuell limitierenden historischen Datentiefe von zwei Monaten liefert die methodisch robuste Arbeit ein skalierbares Fundament für den produktiven Einsatz.

Inhalt

Ehrenwörtliche Erklärung	II
Kurzfassung	III
Inhalt	IV
Abkürzungen	VI
Abbildungen	VII
Tabellen	VIII
1 Projektfokus und Herausforderungen	1
1.1 Themenabgrenzung und Problemstellung	1
1.2 Forschungsfrage und Zielsetzung	1
1.3 Der Ansatz	1
1.4 Herausforderungen	2
2 Theoretische Grundlagen	3
2.1 Zeitreihenanalyse und Autoregression	3
2.2 Multikollinearität und L1/L2-Regularisierung	3
2.3 Ensemble-Algorithmen und Entscheidungsbäume	4
2.4 Data Leakage und Walk-Forward-Validierung	5
2.5 Metriken der Zeitreihen-Folds und Modellgüte	5
3 Methodik und Umsetzung	7
3.1 Datengrundlage	7
3.2 Datenbereinigung und Transformation	7
3.2.1 Autoregressive Filterung und Verhinderung von Leakage	8
3.3 Feature Engineering	9
3.3.1 Zeitliche und kalendarische Transformation	9
3.3.2 Multikollinearitäts-Reduktion meteorologischer Daten	10
3.3.3 Konstruktion autoregressiver Signale	11
3.3.4 Erkenntnisse nach dem Feature-Engineering	13
3.4 Architektur der Vorhersagemodelle	15
3.4.1 Skalierungs- und Validierungslogik	16
3.4.2 Hyperparameter-Tuning	18

3.5	Inferenz und Frontend	20
4	Ergebnisse	23
4.1	Metriken der Modellgüte	23
4.1.1	Modellgüte der Quick-Modelle	23
4.1.2	Relevanz des Quantile Gradient Boosting Regressors	23
4.1.3	Modellgüte der Detailed-Modelle	25
4.2	Feature Decay	26
5	Kritische Reflexion und Ausblick	29
5.1	Zusammenfassung und Reflexion	29
5.2	Ausblick	30
	Literatur	X
	Anhang	XI

Abkürzungen

ACC – Autokorrelationskoeffizient
ACF – Autokorrelationsfunktion
AMD – Advanced Micro Devices
API – Application Programming Interface
CSV – Comma-Separated Values
EN – ElasticNet
ETL – Extract, Transform, Load
GB / GBR – Gradient Boosting Regressor
GYM – Gymnasium (Fitnessstudio)
HTTP – Hypertext Transfer Protocol
IoT – Internet of Things
JSON – JavaScript Object Notation
L1 – L1-Regularisierung (Lasso)
L2 – L2-regularisierung (Ridge)
MSE – Mean Squared Error
NaN – Not a Number
RF – Random Forest
RMSE – Root Mean Squared Error
VIF – Variance Inflation Factor
XAI – Explainable Artificial Intelligence
XGB / XGBoost – Extreme Gradient Boosting

Abbildungen

Abbildung 1: Datenqualität und Zeilenbestand nach der Datenbereinigung	9
Abbildung 2: Pearson-Korrelationsmatrix mit allen Features	10
Abbildung 3: Durchschnittliches Tagesprofil der Auslastung	11
Abbildung 4: Autokorrelationskoeffizienten der Auslastung	12
Abbildung 5: Scatter Plots mit autoregressiven Features	13
Abbildung 6: Variance Inflation Factor Analyse	14
Abbildung 7: Skalierungskonzept zur Vermeidung von Data Leakage	17
Abbildung 8: Stabilitätsanalyse der Detailed-Modelle pro Horizont (R^2)	18
Abbildung 9: Stabilitätsanalyse der Detailed-Modelle pro Horizont (RMSE)	18
Abbildung 10: Sensitivität der Detailed-Modelle bei Parameteränderung	19
Abbildung 11: Effizienz der Detailed-Modelle aller Horizonte	20
Abbildung 12: Dashboard der Inferenz-Demo (Detailed-Modus)	21
Abbildung 13: Dashboard der Inferenz-Demo (Quick-Modus)	22
Abbildung 14: Quantile Gradient Boosting Regressor Demo	24
Abbildung 15: Koeffizienten und Merkmalswichtigkeit der Quick-Modelle	26
Abbildung 16: Koeffizientenverlauf über Zeithorizonte (ElasticNet)	27
Abbildung 17: Bedeutungswandel der Feature-Gruppen (XGBoost)	28

Tabellen

Tabelle 1: Übersicht der ausgewählten Regressionsverfahren	16
Tabelle 2: Metriken der Quick-Modelle	23
Tabelle 3: Metriken der jeweils besten Detailed-Modelle pro Horizont	25

1 Projektfokus und Herausforderungen

1.1 Themenabgrenzung und Problemstellung

Eine präzise und zeitlich hochauflösende Vorhersage der Auslastung von Einrichtungen wie Fitnessstudios für Betreiber oder Kunden stellt eine Herausforderung im Bereich der angewandten Datenanalyse und des maschinellen Lernens dar. Das Besuchsverhalten ist schwer deterministisch zu erfassen, da es von einem Geflecht aus zyklischen Routinen, meteorologischen Einflüssen, gesetzlichen Feiertagen, spontanen Kurzzeiteffekten usw. beeinflusst wird. Um dieses komplexe Geflecht des Besucherverhaltens zu entwirren und dessen Auswirkung auf die Auslastung so akkurat wie möglich abzubilden, entwickelt das vorliegende Forschungsprojekt datengetriebene Prognosemodelle, die es z. B. den Betreibern möglich machen sollen, die Ressourcenplanung, das Personalmanagement, die Klimatisierung oder das Kundenerlebnis zu optimieren.

1.2 Forschungsfrage und Zielsetzung

Die zentrale Forschungsfrage dieses Projektes lautet: Wie präzise lässt sich die Auslastung eines dauerhaft geöffneten Fitnessstudios durch die Kombination von Echtzeit- und externen meteorologischen API-Daten über verschiedene Zeithorizonte durch Machine-Learning-Algorithmen vorhersagen?

1.3 Der Ansatz

Zur Beantwortung der Forschungsfrage, wurde eine Datenpipeline entworfen, die vom Einlesen und Explorieren der Daten über das Modellieren und Validieren bis hin zur Evaluation und Zusammenfassung der Ergebnisse wissenschaftliche Standards befolgt.

Das Projekt verfolgt eine Zwei-Modell-Architektur. Ein Modell, das auf Basis historischer Daten grobe Auslastungsprognosen für den gesamten Tag liefert, sowie ein Multi-Horizon-Modell, das mithilfe von Live-Auslastungsdaten genauere Prognosen für einen Zeitraum von 5 bis 120 Minuten generiert. Der akademische Fokus liegt hierbei auf der XAI und Modelltransparenz sowie der

Generalisierungsfähigkeit der Modelle und der Vermeidung von methodischem Data Leakage.

1.4 Herausforderungen

Die Datenerhebung erfolgt über ein ausfallanfälliges IoT-Gerät, das sich zweier öffentlicher APIs bedient. Daher sind die Rohdaten durch Netzwerkausfälle teilweise unvollständig und müssen durch einen robusten ETL-Prozess für die Zeitreihenanalyse bereinigt werden. Außerdem beschränkt sich die historische Datentiefe auf den Zeitpunkt des Starts der Aufzeichnung am 16. April 2026 um 23:15 Uhr. Dadurch ist der Umfang der Daten zum Zeitpunkt der Forschung und Entwicklung im Projekt auf ca. 48 Tage begrenzt und deckt nur den Frühling sowie den Anfang des Frühsommers ab. Sollte die Datenerhebung länger als ein Jahr andauern, um die Forschung mit einem Datensatz fortzuführen, der das gesamte Jahr abdeckt, greift ein gleitendes Fenster vor der Modellierung. Dieses begrenzt die Datentiefe auf 365 Tage, um langfristige Verhaltensänderungen (Concept Drift) abzufangen. Während diese Restriktion die Genauigkeit der Modelle mittelfristig verbessert, schränkt sie die Modelle darin ein, langjährige saisonale Verschiebungen oder sich wiederholende Feiertageffekte über Jahre hinweg zu erlernen.

2 Theoretische Grundlagen

Das methodische Vorgehen des Projektes stützt sich auf etablierte Konzepte der künstlichen Intelligenz, des überwachten maschinellen Lernens und der Zeitreihenanalyse, um hochdimensionale Zeitreihendaten im Kontext der Auslastung in verständliche Zusammenhänge umzuwandeln.

2.1 Zeitreihenanalyse und Autoregression

Eine Zeitreihe ist als eine sequenzielle Abfolge von Beobachtungen definiert, die in äquidistanten Intervallen gemessen werden. Die Zeitreihenzerlegung zerlegt das Signal typischerweise in eine Trendkomponente für langfristige Entwicklungen, eine saisonale Komponente und eine Restkomponente für stochastisches Rauschen.¹

In autoregressiven Modellen werden historische Werte der Zielvariable (sog. Lags) als direkte Prädiktoren für zukünftige Ausprägungen herangezogen.²

Die Autokorrelationsfunktion (ACF) nach Pearson ist für das Projekt das zentrale statistische Werkzeug, um festzustellen, wie stark eine Beobachtung mit ihren eigenen vergangenen Werten korreliert, um optimale Lags für das Feature Engineering zu bestimmen.³

2.2 Multikollinearität und L1/L2-Regularisierung

In einem hochdimensionalen Datensatz korrelieren Prädiktoren häufig stark miteinander (z. B. Temperatur mit gefühlter Temperatur). Diese Multikollinearität destabilisiert die Matrixinversion in klassischen linearen Modellen, da die Determinante fast null wird, und führt zu einer starken Varianz der Regressionskoeffizienten. Die offensichtlichsten multikollinearen Prädiktorenkombinationen kann man an den Werten nahe +1 und -1 in einer Korrelationsmatrix erkennen, die jeweils 2 Prädiktoren miteinander vergleicht. Um eventuell verdeckte Redundanzen aufzudecken, kann man den Variance Inflation Factor (VIF) des jeweiligen Prädiktors berechnen. Je größer der VIF ist, desto stärker korreliert der Prädiktor mit der Kombination aus mehreren

¹ Vgl. (Brockwell & Davis, 2016), S.20

² Vgl. ebd., S.91

³ Vgl. ebd., S.13f., S.16

Prädikatoren. Übersteigt der VIF spezifische Schwellen, wie etwa $VIF > 10$, signalisiert das eine kritische Kollinearität, die die Modellinterpretation gefährdet.⁴

Dieser Instabilität der klassischen linearen Modelle bei Multikollinearität kann man mit Regularisierungsverfahren entgegenwirken. Die Ridge-Regression fügt der Methode der kleinsten Quadrate einen L2-Strafterm ($\lambda \sum \beta_j^2$) hinzu und erzwingt damit, dass Koeffizienten korrelierter Variablen gleichmäßig in Richtung Null schrumpfen. Das senkt die Varianz und macht das Modell stabiler.⁵

Die Lasso-Regression nutzt einen L1-Strafterm ($\lambda \sum |\beta_j|$), der die absoluten Werte unwichtiger Koeffizienten auf Null setzen kann. Dadurch werden Variablen über eine implizite Merkmalsauswahl aus dem Modell gelöscht und es bleiben nur noch die wichtigen Variablen für das Modell übrig. Das macht das Modell ebenfalls stabiler und aussagekräftig.⁶

2.3 Ensemble-Algorithmen und Entscheidungsbäume

Regularisierte lineare Modelle bilden zwar eine gute Basis, allerdings weisen Daten aus und um das Fitnessstudio herum hochgradig nichtlineare Interaktionseffekte auf. Der Einfluss eines Wochentages ist unter anderem anders zu bewerten, wenn ein gesetzlicher Feiertag auf diesen fällt oder nicht. Um diese abzubilden, werden baumbasierte Ensemble-Verfahren herangezogen.

Der Random Forest kombiniert durch Bootstrap Aggregating (Bagging) viele unkorrelierte Entscheidungsbäume miteinander. Jeder dieser Bäume lernt auf einer zufälligen Teilmenge der Daten und Merkmale, wodurch das Rauschen in der Modellierung minimiert wird.⁷

Gradient Boosting Algorithmen wie XGBoost gehen noch einen Schritt weiter und nutzen sogenannte schwache Basislerner, die iterativ und sequenziell zu

⁴ Vgl. (James, Witten, Hastie, Tibshirani, & Taylor, 2023), S.106-109

⁵ Vgl. ebd., S.240, S.244

⁶ Vgl. ebd., S.244

⁷ Vgl. ebd., S.346f.

optimierten Bäumen kombiniert werden, die verbleibende Fehler (Residuen) der Vorgänger minimieren.⁸ Wie Chou und Tran (2018) in der Literatur belegen, weisen Boosting und Bagging Leistungsvorteile bei der Modellierung von Gebäudeauslastung und komplexen Zeitreihen auf.⁹

2.4 Data Leakage und Walk-Forward-Validierung

In der ETL sowie bei der Modellierung prädiktiver Zeitreihenmodelle ist das sogenannte Data Leakage ein zentraler Faktor für die Modellgüte. Gelangen unzulässigerweise Informationen aus der Validierungsmenge in den Trainingsprozess, spricht man von Data Leakage. Bei Zeitreihen verursachen unter anderem die Bildung des arithmetischen Mittelwerts oder eine rein zufällige Mischung durch Resampling-Techniken wie die k-fold Cross-Validation Data Leakage. Dadurch lernen Modelle Kausalitäten, die zum Zeitpunkt der eigentlichen Vorhersage nicht existieren würden.^{10 11}

Um das zu vermeiden, wird die Walk-Forward-Validierung (TimeSeriesSplit in Python) angewendet. Hierbei wird der Datensatz chronologisch geordnet, das Modell auf einem historischen Fenster trainiert und ausschließlich auf dem darauffolgenden Zeitfenster validiert. Die Berechnung des Mittelwertes (μ) und der Standardabweichung (σ) für die Skalierung erfolgt, so wie das Modelltraining, nur auf dem Trainingsfenster.¹² Ohne diese Maßnahmen wäre die Modellgüte bei realen Daten durch Overfitting negativ beeinflusst.¹³

2.5 Metriken der Zeitreihen-Folds und Modellgüte

Um die Vorhersagegenauigkeit von Modellen in der Modellierung sowie der Inferenz bewerten und vergleichen zu können, werden spezifische Metriken berechnet. Da die Prognose der Fitnessstudioauslastung ein Regressionsproblem darstellt, werden primär absolute Fehlermaße sowie relative Gütemaße zur Beurteilung verwendet.

⁸ Vgl. (James, Witten, Hastie, Tibshirani, & Taylor, 2023), S.347f.

⁹ Vgl. (Jui-Sheng & Duc-Son, 2018), S.724

¹⁰ Vgl. (James, Witten, Hastie, Tibshirani, & Taylor, 2023), S.206-208

¹¹ Vgl. (Kapoor & Narayanan, 2023), S.4, S.8

¹² Vgl. ebd., S.4 L3.1 und L1.2

¹³ Vgl. ebd., S.1, S.4, S.8

Theoretische Grundlagen

Der Mean Squared Error (MSE) berechnet den Durchschnitt der quadrierten Differenzen zwischen prognostizierten und tatsächlichen Auslastungswerten. Durch das Quadrieren der Residuen werden große Abweichungen überproportional stark gewichtet. In der Praxis dient der MSE häufig als zu minimierende Verlustfunktion während der Hyperparameteroptimierung. Zur besseren Interpretierbarkeit des absoluten Vorhersagefehlers wird der Root Mean Squared Error (RMSE) verwendet. Als Quadratwurzel des MSE transformiert der RMSE den Fehlerwert zurück in die ursprüngliche Skalierung der Zielvariable. Je geringer der Wert, desto besser ist die Vorhersagegüte.¹⁴

Das Bestimmtheitsmaß (R^2) liefert eine statistische Aussage darüber, welcher Anteil der Varianz der abhängigen Zielvariable durch die unabhängigen Prädiktoren des Modells erklärt wird. Der R^2 -Score kann für den direkten Vergleich der Modellgüte unter den Modellen herangezogen werden. Ein R^2 -Score nahe 1 signalisiert hierbei eine hohe Modellgüte.¹⁵

Der arithmetische Mittelwert (μ) sowie die Standardabweichung (σ) der jeweiligen Metriken werden im Rahmen der TimeSeriesSplit-Kreuzvalidierung verwendet. Die R^2 -Werte der Zeitreihen-Folds werden gemittelt, um die durchschnittliche Vorhersageleistung des Modells zu berechnen, während die Standardabweichung dessen Stabilität über alle Folds angibt.¹⁶

¹⁴ Vgl. (James, Witten, Hastie, Tibshirani, & Taylor, 2023), S.28f.

¹⁵ Vgl. ebd., S.78f.

¹⁶ Vgl. ebd., S.206

3 Methodik und Umsetzung

Die Systemarchitektur des Projektes wurde modular implementiert und organisiert. Je ein isolierter Teilbereich vom ETL-Prozess bis zur Inferenz. Sie umfasst die Datenbeschaffung, eine Bereinigungspipeline, XAI-Automatisierung für die explorative Analyse, ein detailliertes Feature-Engineering, eine Multi-Modell-Trainingspipeline mit Hyperparameter-Tuning und Gittersuche sowie eine asynchrone Inferenz-Engine für eine Demo.

3.1 Datengrundlage

Das Extrahieren der Rohdaten für die ETL und Modellierung erfolgte über ein Python-Skript, das in einem rootless Podman-Quadlet-Container mit automatischem User-Namespace auf einem Raspberry Pi 5 alle fünf Minuten über einen Cron-Job ausgeführt wird. Dieses Python-Skript sendet Anfragen an zwei öffentliche APIs und erhält Antworten im JSON-Format. Die Daten zur Zielvariable, der Fitnessstudioauslastung, werden von einer öffentlichen API abgerufen, die von der mobilen Anwendung des Fitnessstudios genutzt wird, und werden zusätzlich mit lokalen meteorologischen Daten der Open-Meteo-API angereichert. Zusammen mit Zeitstempeln werden die Rohdaten in einer fortlaufenden CSV-Datei gespeichert. Die Auswahl der gesammelten Merkmale ist hierbei nicht auf das Projekt zugeschnitten, da die Datengewinnung einen Monat vor dem Projekt startete.

3.2 Datenbereinigung und Transformation

Die Rohdaten weisen systembedingt Inkonsistenzen auf. Netzwerkausfälle, Latenzen beim API-Aufruf oder Serverneustarts führen zu asynchronen Zeitstempeln und fehlenden Werten. Dabei beschränken sich fehlende Werte im Beobachtungszeitraum ausschließlich auf meteorologische Daten. Die Bereinigungspipeline im ETL-Modul bereitet die Rohdaten für das Feature-Engineering sowie Zeitreihenmodelle vor und generiert anschließend einen detaillierten Bericht, der zentrale Eckdaten zur Datenqualität, historische Auslastungsspitzen sowie meteorologische Wertebereiche usw. aggregiert.

Im ersten Schritt wird der Datensatz mithilfe der Pandas-Bibliothek auf ein lückenloses 5-Minuten-Raster gezwungen. Dabei entstehende oder schon vorhandene Datenlücken werden datentypspezifisch imputiert: Kontinuierliche numerische Variablen, wie die Temperatur, werden linear interpoliert. Für binäre Zustandsvariablen, wie die Aussage, ob Tageslicht vorhanden ist, wird der letzte bekannte Wert für die Imputation verwendet (Forward-Fill-Verfahren). Um dem Modell transparent zu vermitteln, welche Werte imputiert und welche reale Zustände sind, werden imputierte Zeilen mit einer binären Kontroll-Flag („is_imputed“: 1) markiert und die zusammenhängende Imputationsperiode numerisch in 5-Minuten-Schritten dokumentiert.

3.2.1 Autoregressive Filterung und Verhinderung von Leakage

Hält die Imputationsperiode länger als 24 5-Minuten-Zeitschritte (zwei Stunden) an, werden die Zeilen ab dem 25. Zeitschritt sowie die darauffolgenden zwei Stunden nach Ende der Lücke aus dem Datensatz gelöscht, um Overfitting durch „Interpolation Poisoning“ und Data Leakage durch verfälschte autoregressive Features zu vermeiden. Fällt der Datenlogger für längere Zeit aus, erzeugt die lineare Interpolation fehlerfreie Steigungslinien zwischen dem letzten bekannten und dem neuen Wert, die ein Modell lernen und übermäßig auf das Imputationsverfahren anpassen würde. Daher wird der imputierte Zeitraum aus dem Datensatz gelöscht.

Die darauffolgenden zwei Stunden werden aufgrund autoregressiver Zeitreihen-Features, die auf Basis interpolierter Werte des vorherigen Zeitraums berechnet wurden, ebenfalls gelöscht. Ein gleitendes 2-Stunden-Fenster identifiziert Zeitstempel, die sich innerhalb der zwei Stunden nach einer signifikanten Lücke befinden, und überschreibt deren Zeitreihen-Prädiktoren explizit mit „NaN“. So wird garantiert, dass Zeitreihen-Features wie Trendangaben oder Lags niemals auf interpoliertem Rauschen aufbauen.

Anhand von Abbildung 1 sind implizit die Qualität des Rohdatensatzes sowie der konkrete Zeilenbestand für die Modellierung nach der Bereinigung des Datensatzes zu erkennen.

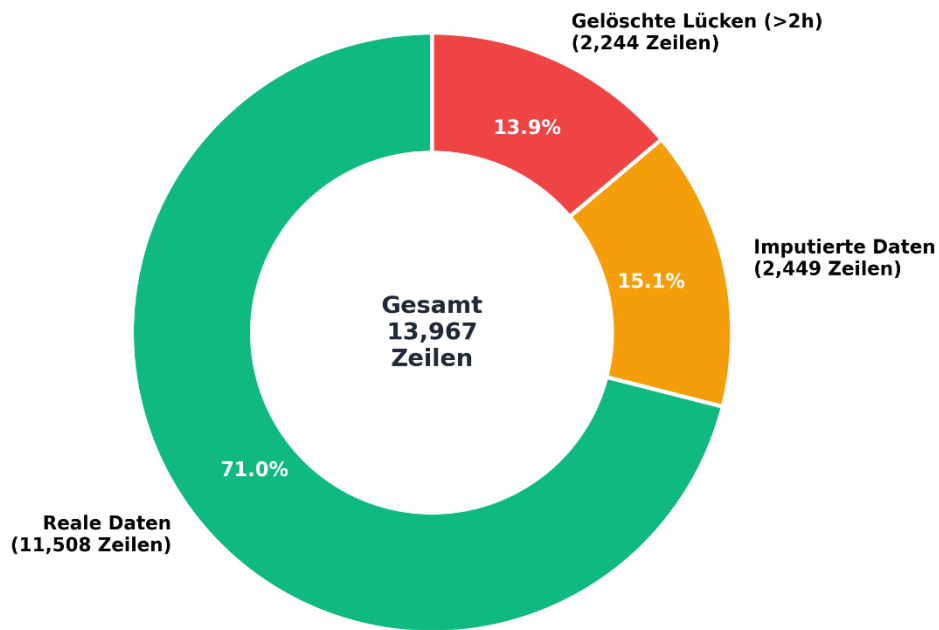


Abbildung 1: Datenqualität und Zeilenbestand nach der Datenbereinigung

3.3 Feature Engineering

3.3.1 Zeitliche und kalendarische Transformation

Da die Uhrzeit zyklisch verläuft, Modelle den Abstand von 23:55 Uhr zu 00:00 Uhr allerdings numerisch als riesig interpretieren, muss man bei der Datenanalyse die Uhrzeit über Sinus- und Kosinus-Funktionen als Koordinaten auf einem Kreisbogen encodieren, um periodische Zeitstrukturen kontinuierlich und sprungfrei in Machine-Learning-Algorithmen abzubilden. Aus dem Zeitstempel werden zusätzlich Wochen- und Feiertags-Features extrahiert. Mittels One-Hot-Encoding wird jeweils eine binäre Spalte pro Wochentag erstellt. Um Effekte durch Feiertage oder Schulferien einzufangen, integriert die Pipeline außerdem die OpenHolidays-API sowie die holidays Python-Bibliothek, um die Zeitstempel mit den Daten der Schnittstellen abzugleichen. Anschließend werden die betroffenen Tage wieder über One-Hot-Encoding separat binär markiert. Um Latenzen und Rate-Limits bei der OpenHolidays-API im Produktivbetrieb zu vermeiden, wurde zudem ein lokaler JSON-basierter Cache für Datumsbereiche der Ferien implementiert.

3.3.2 Multikollinearitäts-Reduktion meteorologischer Daten

In der deskriptiven Analyse während der Implementierung der ETL-Pipeline offenbarte sich, dass die Außentemperatur und die gefühlte Temperatur nahezu identische Varianzen aufwiesen. Diese Korrelation war mit einem Korrelationskoeffizienten von 0,97 stark in der Korrelationsmatrix sichtbar. Um die Kollinearität zu eliminieren und die Regressionskoeffizienten zu stabilisieren, wurde das Feature der gefühlten Temperatur durch die Differenz von Temperatur und gefühlter Temperatur ersetzt. Das neue Feature „temp_diff“ fängt mit einem maximalen VIF-Wert von 4,4 implizit den Effekt von schwülem Wetter oder starkem Windchill ein, ohne Multikollinearität zu verursachen, wie man in Abbildung 2 sehen kann.

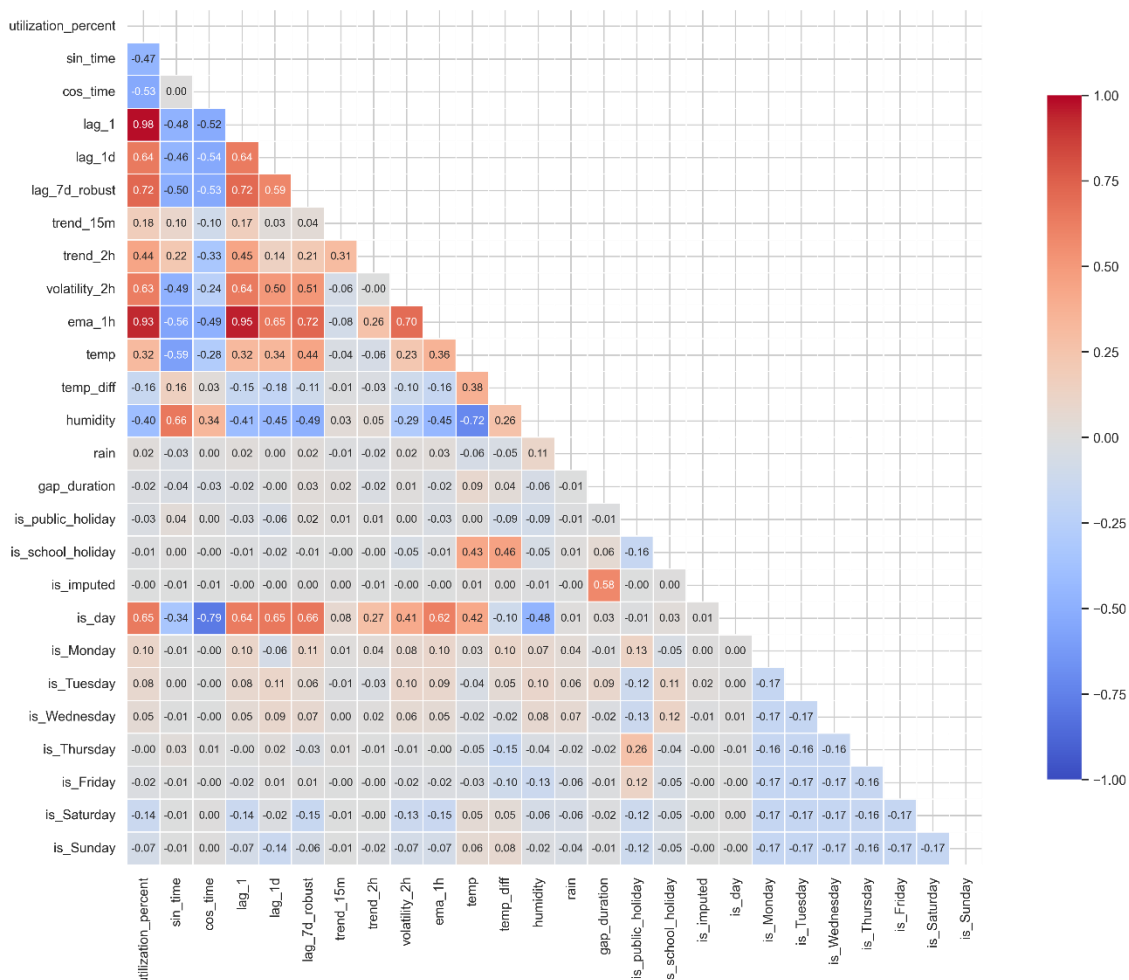


Abbildung 2: Pearson-Korrelationsmatrix mit allen Features

3.3.3 Konstruktion autoregressiver Signale

Wie in Abbildung 3 zu erkennen ist, unterliegt die Auslastung im durchschnittlichen Tagesverlauf einer starken Saisonalität über den Beobachtungszeitraum von knapp zwei Monaten. Diese Eigenschaft legt nahe, dass vergangene Zustände eine hohe prädiktive Aussagekraft für zukünftige Werte besitzen, und begründet die Konstruktion autoregressiver Features, wie Tages- und Wochen-Lags, um die Auslastungskurve präziser zu prognostizieren. Hierbei dürfen Werte der Wochen-Lags, die in den Zeitraum von Feiertagen oder Schulferien fallen, nicht als Prädiktor für reguläre Tage verwendet werden, da abnormale Werte die Prognose für diese kontaminieren würden. Daher iteriert die Pipeline in 7-Tages-Schritten so lange rückwärts in der Historie, bis sie auf einen validen Tag stößt.

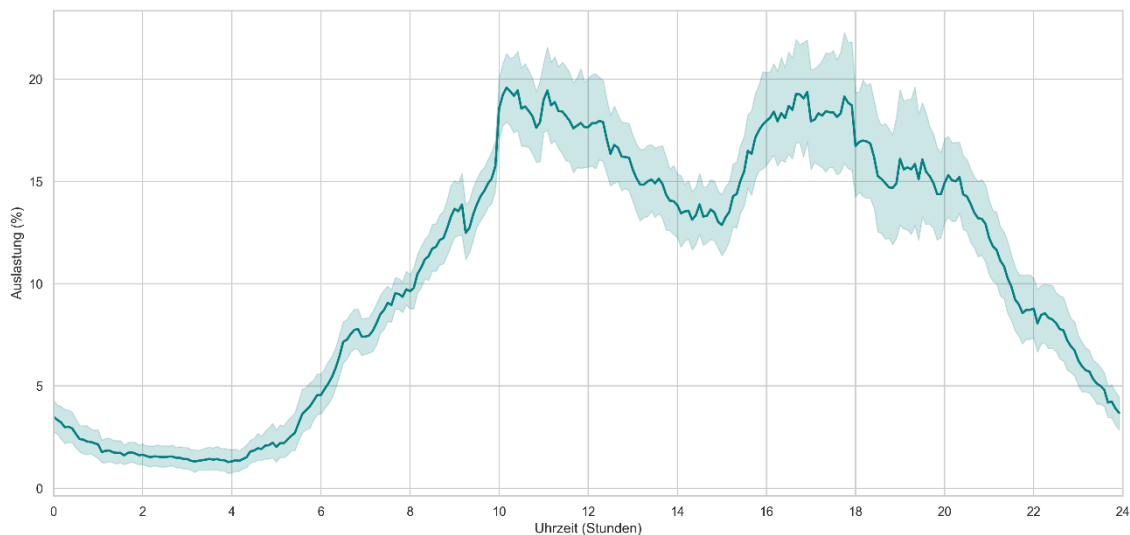


Abbildung 3: Durchschnittliches Tagesprofil der Auslastung

Des Weiteren wird die zeitliche Trägheit der Auslastung für präzise Prognosen über kurzfristige Lags mithilfe der Standard Pearson Autokorrelationsfunktion (ACF) untersucht. Die ACF nimmt eine zeitliche Verzögerung (Lag) h entgegen und liefert einen Autokorrelationskoeffizienten im Bereich von -1 bis 1, der aussagt, wie relevant die Auslastung vor h Minuten für die aktuelle Auslastung ist. Wiederholt man diesen Prozess für alle Lags in einem bestimmten Zeitraum, wird die zeitliche Trägheit der Auslastung in einem Stabdiagramm wie in Abbildung 4 sichtbar.

Die im Projekt verwendete Autokorrelationsfunktion:¹⁷

$$\hat{\rho}(h) = \frac{\hat{\gamma}(h)}{\hat{\gamma}(0)} = \frac{\sum_{t=1}^{n-|h|} (x_{t+|h|} - \bar{x})(x_t - \bar{x})}{\sum_{t=1}^n (x_t - \bar{x})^2}$$

$\hat{\rho}(h)$ = Autokorrelationsfunktion

h = betrachtete zeitliche Verzögerung (Lag)

$\hat{\gamma}(h)$ = Autokovarianz bei Lag h

$\hat{\gamma}(0)$ = Gesamtvarianz der Zeitreihe

n = Gesamtanzahl der Beobachtungen

x_t und $x_{t+|h|}$ = beobachtete Werte zu den Zeitpunkten t und $t + |h|$

$\bar{x}$ = Mittelwert der Zeitreihe

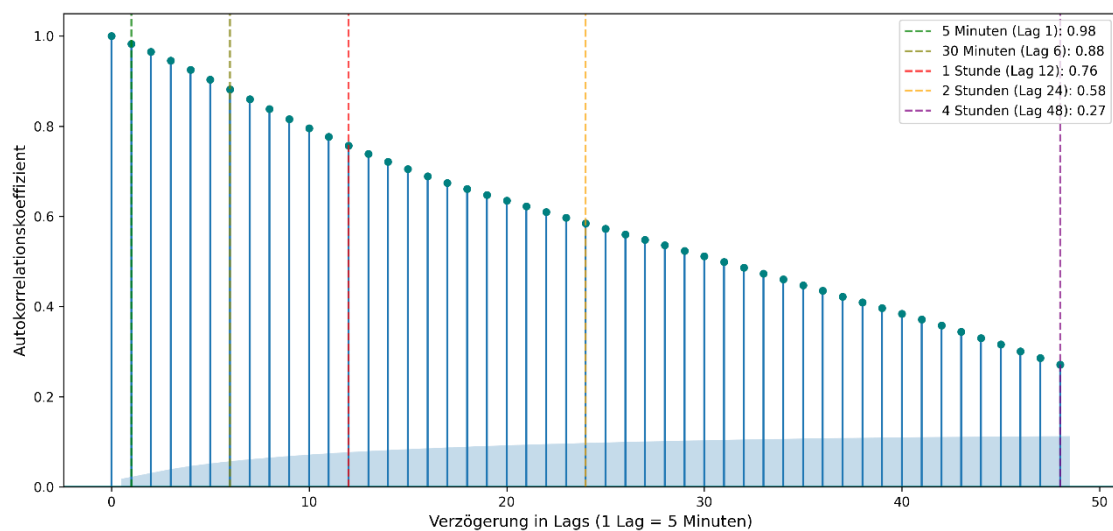


Abbildung 4: Autokorrelationskoeffizienten der Auslastung

Auf Basis der Autokorrelationsfunktion, evaluiert über 48 Lags (4 Stunden), lassen sich folgende Erkenntnisse gewinnen und kurzfristige, autoregressive Features konstruieren: Neben der hohen statistischen Vorhersagekraft der Auslastungswerte von vor 5 Minuten („lag_1“) mit einem ACC von 0,98 sind die Lags 3, 12 und 24 ebenfalls relevante Punkte für die Konstruktion autoregressiver Features. Alle Punkte nach Lag 24 (ACC von 0,58) werden für das Feature Engineering für kurzfristige autoregressive Signale nicht berücksichtigt, da der ACC nach zwei Stunden stark abfällt. Mithilfe der Lags 3

¹⁷ Vgl. (Brockwell & Davis, 2016), S.16

und 24 werden kurz- und langfristige Trends („trend_15m“, „trend_2h“) über die Steigung berechnet.

Lag 24 wird außerdem für die Berechnung der Auslastungsschwankung („volatility_2h“) verwendet und ein exponentiell geglätteter Durchschnitt („ema_1h“) dämpft kurzfristiges Rauschen ab. Ein Blick auf die Autokorrelationskurve verrät, dass die Auslastung innerhalb von fünf Minuten kaum abnimmt, die Trägheit der Auslastung allerdings nach spätestens zwei Stunden stark abnimmt. Dieses Zeitfenster ist der ideale Zeitraum und Grundlage für die Konstruktion statistisch vorhersagekräftiger autoregressiver Features.

3.3.4 Erkenntnisse nach dem Feature-Engineering

Mit dem Abschluss des Feature-Engineerings, lassen sich neue, aussagekräftige Beziehungen und Effekte visualisieren:

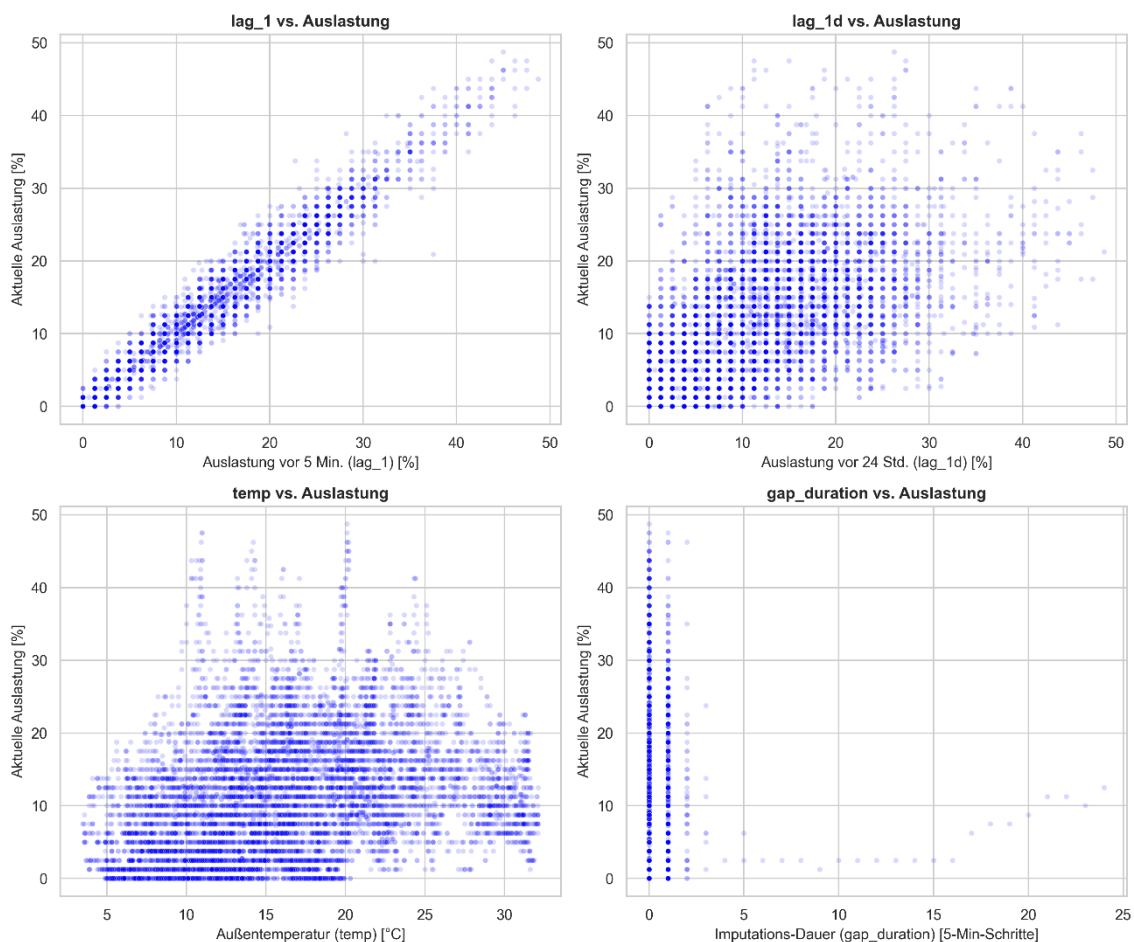


Abbildung 5: Scatter Plots mit autoregressiven Features

Anhand des Graphen oben links in der Abbildung 5, der die aktuelle Auslastung mit dem Wert von vor fünf Minuten vergleicht, erkennt man die stark lineare Verteilung der Datenpunkte, was die starke kurzfristige Trägheit der Auslastung unterstreicht. Der Graph rechts daneben zeigt, dass die Auslastung von vor 24h zwar weniger stark mit der aktuellen Auslastung korreliert, allerdings weiterhin ein linearer Trend zu erkennen ist. Der Graph unten links zeigt entgegen der Erwartung keinen klaren Abwärtstrend mit steigender Temperatur, was allerdings dem kurzen Beobachtungszeitraum geschuldet sein kann, der wenig extreme Temperaturspitzen enthält. Der Graph unten rechts sagt aus, dass, wenn die Wetter-API ausfällt und zu imputierende Lücken im Datensatz erzeugt, die Lücken nur selten groß sind, was für die Qualität des Datensatzes und der Datengrundlage spricht.

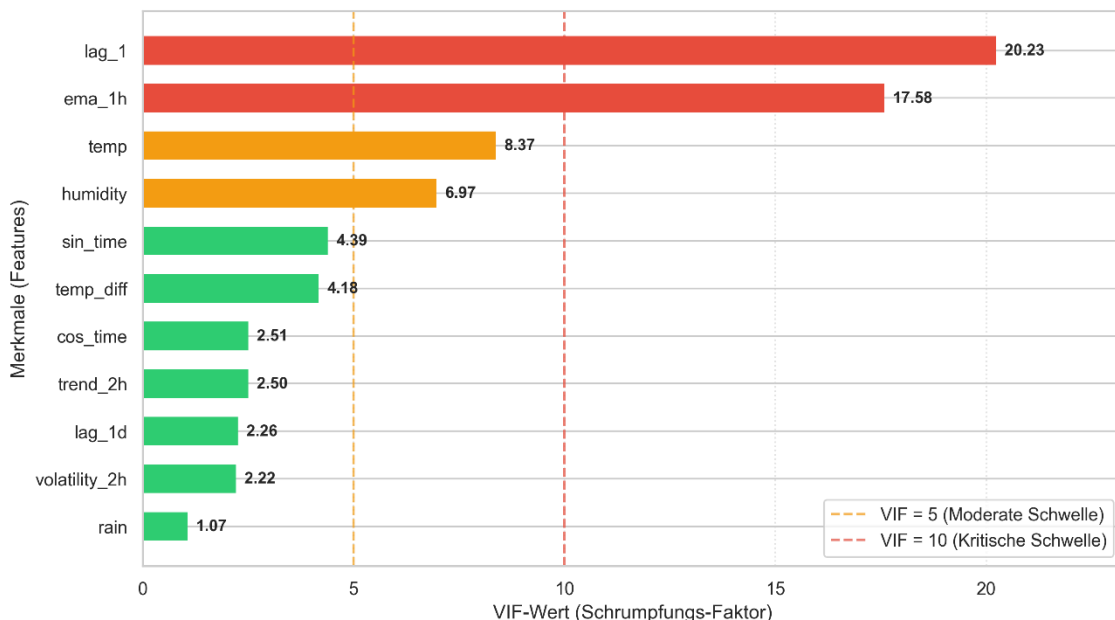


Abbildung 6: Variance Inflation Factor Analyse

Die hohen VIF-Werte der zeitlichen Features in Abbildung 6 erklären sich nicht durch schlechtes Feature-Engineering, sondern durch die Eigenschaften von Zeitreihen und Zeitreihen-Features. „Lag_1“ ist ein direkter mathematischer Bestandteil der Formel von „ema_1h“. Die hohen VIF-Werte beweisen lediglich, dass die Features die zeitliche Trägheit korrekt abgebildet haben. Beginge man den Fehler, eines der beiden Features zu entfernen, um Lehrbuchregeln zu entsprechen, verliere man wichtige Informationen (Underfitting).

Das Projektziel ist zudem rein prädiktiv und nicht inferenzstatistisch, was der Multikollinearität weniger Gewicht zuspricht, solange lineare Beziehungen über die Zeit stabil bleiben. Die Verwendung der L1- und L2- Strafterme sowie multikollinearitätsresistenter, baumbasierter Modelle schützt zusätzlich vor negativen Einflüssen der Multikollinearität auf die Vorhersagegüte.

3.4 Architektur der Vorhersagemodelle

Die Trainings-Pipeline des Projektes teilt die Architektur in zwei fundamentale Ansätze auf:

1. Das „Quick-Modell“: Die Modellfamilie der „Quick-Modelle“ verwendet ausschließlich deterministische Kalenderdaten und Wetter-Features für die Vorhersage. Das Modell wird ohne Lag-Features auf den historischen Daten trainiert und ist für langfristige Simulationen prädestiniert, wie die Beantwortung der Frage: „Welche Auslastung ist am kommenden Dienstag um 18:00 Uhr zu erwarten, sofern es stark regnet?“
2. Die „Detailed-Modelle“ (Multi-Horizon): Die Detailed-Modelle sind wesentlich komplexer als das Quick-Modell. Sie nutzen alle Features samt autoregressiver Lags. Da die Relevanz aussagekräftiger kurzfristiger Lags stark abnimmt, je weiter man in die Zukunft prognostizieren möchte, wurde auf ein einzelnes Modell verzichtet. Stattdessen trainiert das System sechs voneinander isolierte Algorithmen für die Zeithorizonte von 5, 15, 30, 60, 90 und 120 Minuten. Dadurch sind Detailed-Modelle auf einen Vorhersagehorizont von zwei Stunden begrenzt, während Quick-Modelle die Auslastungskurve des z. B. gesamten Tages prognostizieren können. Dafür liefern Detailed-Modelle kurzfristig genauere Prognosen.

Diese Architektur ergibt sieben Modellfamilien, für die jeweils 7 Modelle trainiert und miteinander verglichen werden. Insgesamt werden 49 Modelle von der Trainings-Pipeline erstellt. Die Tabelle 1 zeigt auf, welche Algorithmen die Trainings-Algorithmen für die Modellierung verwenden. Es wird jeweils nur das Modell pro Modellfamilie verwendet, das im Vergleich anhand konkreter Metriken am besten abschneidet.

Modellname	Modelltyp
Ridge Regression (L2)	Linear
Lasso Regression (L1)	Linear
ElasticNet Regression (L1 & L2)	Linear
Random Forest Regressor	Baumbasiert
Gradient Boosting Regressor	Baumbasiert
XGBoost Regressor	Baumbasiert
Quantile Gradient Boosting Regressor	Baumbasiert

Tabelle 1: Übersicht der ausgewählten Regressionsverfahren

3.4.1 Skalierungs- und Validierungslogik

Die Vorverarbeitung geschieht über einen Scikit-Learn ColumnTransformer. Die Daten müssen auf einen einheitlichen Maßstab gebracht werden, damit das Modell nicht von Variablen mit großen Wertebereichen dominiert wird. Kontinuierliche numerische Variablen werden durch einen StandardScaler standardisiert, sodass sie einen Mittelwert (μ) von 0 und eine Standardabweichung (σ) von 1 aufweisen. Die Zielvariable sowie binäre Variablen bleiben unskaliert.

Die Skalierung unterliegt dabei strengen Vorgaben, um Data Leakage zu vermeiden. Wie Abbildung 7 zeigt, berechnet der Skalierer ausschließlich den Mittelwert und die Standardabweichungen anhand der Trainings-Folds (Train), um diese Transformation anschließend auf den Validierungs-Folds (Val) anzuwenden. Dadurch erhalten die Modelle im Training keine impliziten Hinweise darauf, wie sich die Daten in der Zukunft verhalten.

Insgesamt wird der Datensatz im TimeSeriesSplit-Verfahren dieses Projektes in fünf Folds mit chronologisch anwachsender Länge aufgeteilt. Während der erste Durchlauf (Trainings- und Validierungsfenster) das erste Fünftel der Daten umfasst, wächst das Trainingsfenster im zweiten Durchlauf um die historischen Daten des vorherigen Validierungsfensters an. Somit deckt der zweite Fold insgesamt zwei Fünftel des Gesamtdatensatzes ab, und so weiter.

Das Modell wird auf jedem Fold isoliert trainiert und validiert. Am Ende jedes einzelnen Durchlaufs werden das Bestimmtheitsmaß (R^2) sowie der absolute Vorhersagefehler (RMSE bzw. MSE) berechnet.

Nach Abschluss aller fünf Durchläufe werden diese Metriken über alle Folds hinweg gemittelt und ihre jeweilige Standardabweichung wird berechnet, um die Gesamtleistung sowie die Stabilität der Modelle zu evaluieren.

Das leistungsstärkste Modell wird anschließend anhand des höchsten Bestimmtheitsmaßes (R^2) identifiziert.

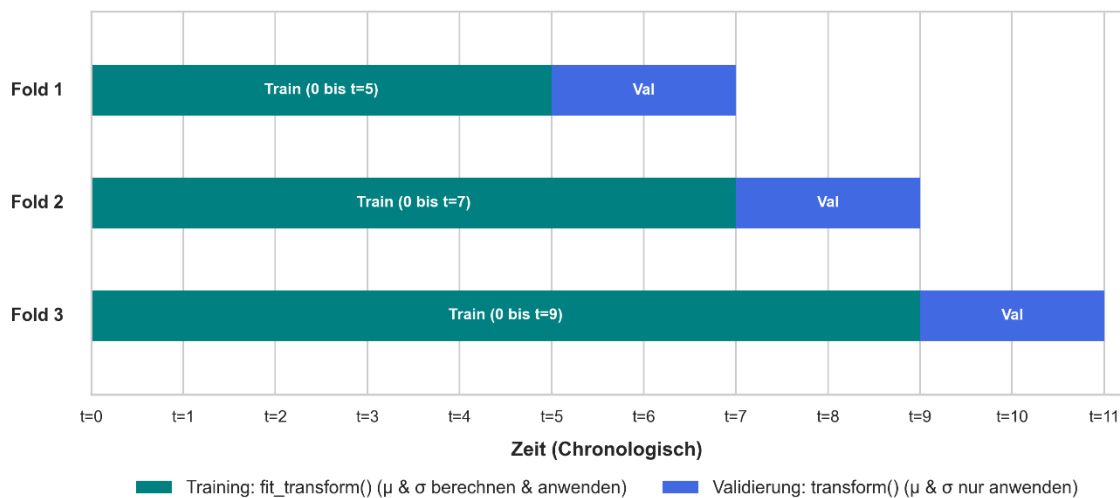


Abbildung 7: Skalierungskonzept zur Vermeidung von Data Leakage

Wie sich die Modellstabilität über verschiedene Zeitfenster und Vorhersagehorizonte verhält, visualisieren Abbildung 8 und 9 am Beispiel der Detailed-Modelle. Sie zeigen, dass die Vorhersagefehler (RMSE) über alle Folds hinweg stabil bleiben. Das ist ein Nachweis dafür, dass die Modelle keine Ausreißer auswendig gelernt haben und das präventive Skalierungskonzept erfolgreich implementiert wurde.

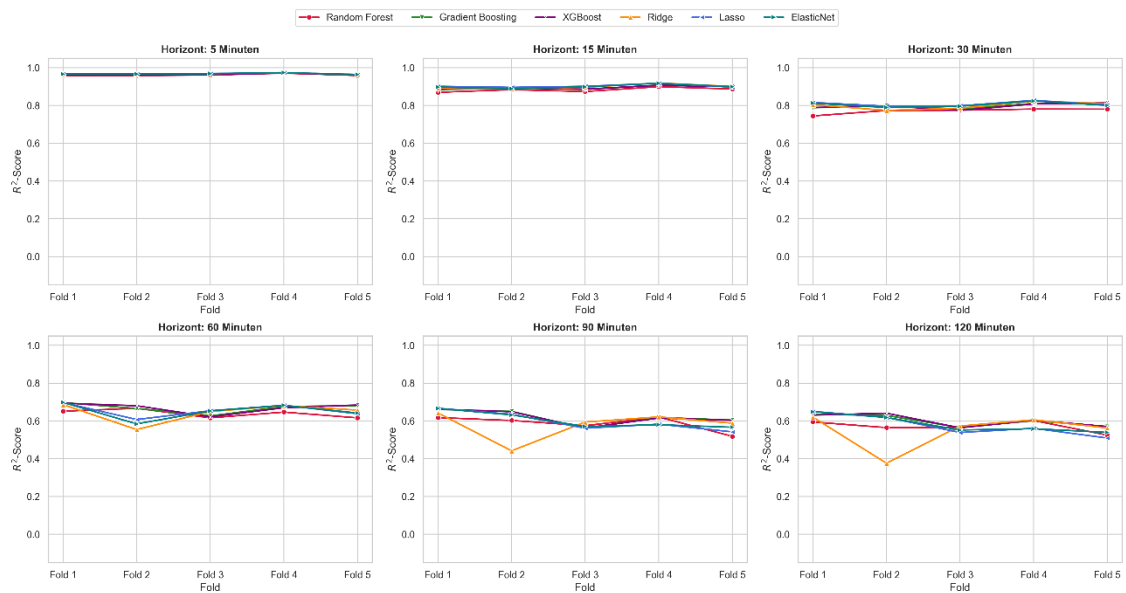


Abbildung 8: Stabilitätsanalyse der Detailed-Modelle pro Horizont (R^2)

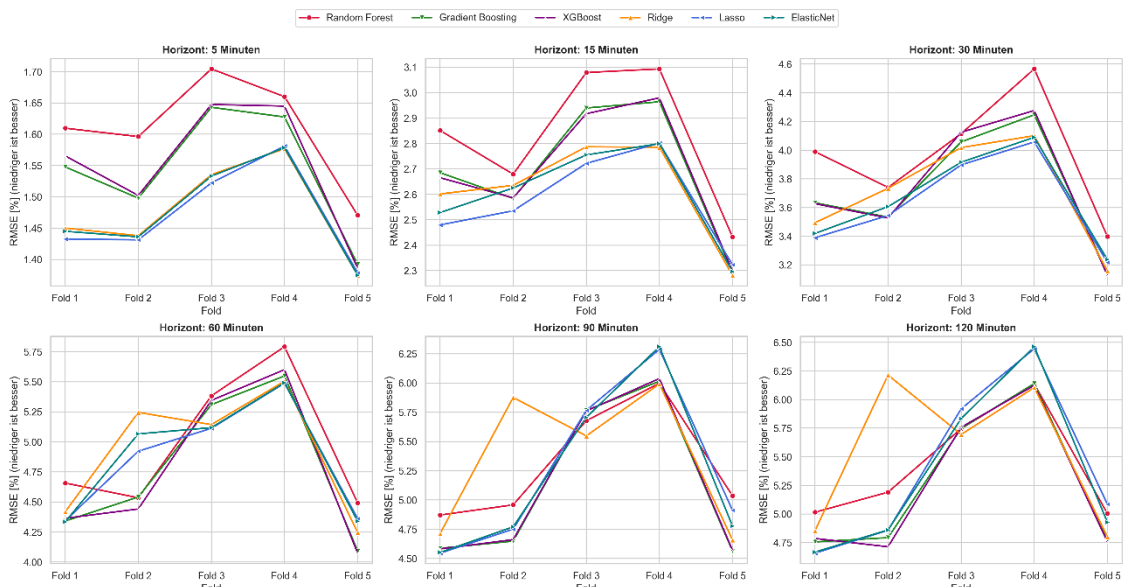


Abbildung 9: Stabilitätsanalyse der Detailed-Modelle pro Horizont (RMSE)

3.4.2 Hyperparameter-Tuning

Um die optimale Konfiguration der Modelle zu finden, wurde eine systematische Grid-Search über einen definierten Suchraum durchgeführt. Im Gegensatz zur Leistungsbewertung der Modelle über das gemittelte Bestimmtheitsmaß (R^2) in der Kreuzvalidierung des vorherigen Kapitels, erfolgt die interne Optimierung

der Hyperparameter während der Grid-Search über die absolute Fehlerminimierung (MSE). Da ein vollständiger Durchlauf bei 49 Modellen bei einer Parallelisierung auf 28 logischen Kernen (AMD Ryzen 9 9950x3D) ca. 8 Minuten Rechenzeit beansprucht, wurde der Prozess modularisiert: Über einen Boolean-Schalter ermöglicht die Pipeline, die Grid-Search zu überspringen und stattdessen die besten Parameter aus einem JSON-Cache zu laden. Ein weiterer Boolean-Schalter bestimmt, wie groß der Suchraum für die Grid-Search ist. Auch innerhalb der Grid-Search wird das TimeSeriesSplit-Verfahren verwendet, um Data Leakage zu vermeiden.

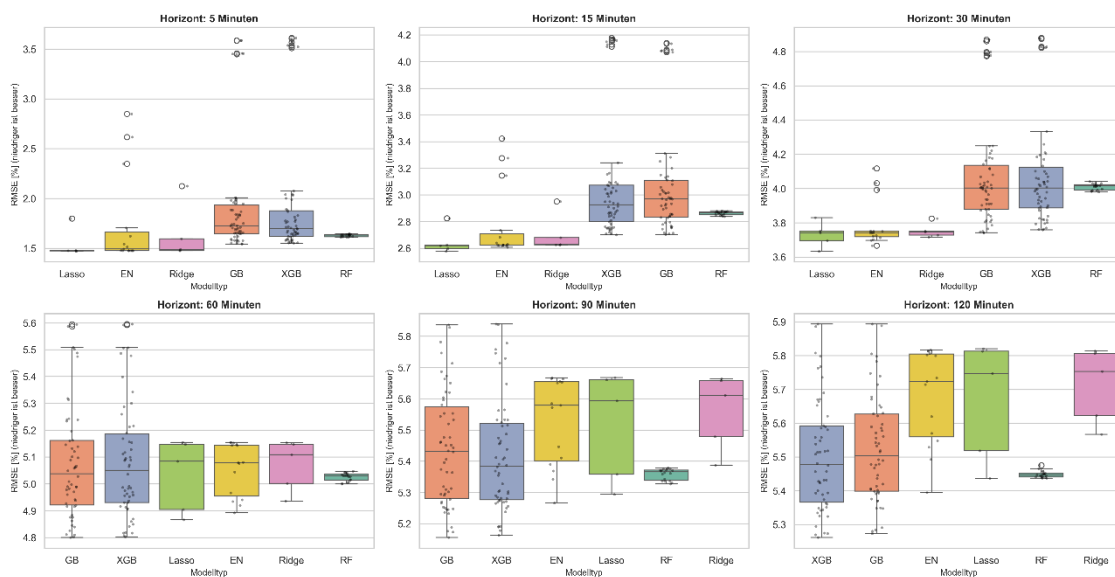


Abbildung 10: Sensitivität der Detailed-Modelle bei Parameteränderung

Die Boxplot-Analyse in Abbildung 10 belegt, dass baumbasierte Ensemble-Methoden sensibel auf Änderungen der Hyperparameter reagieren und suboptimale Konfigurationen zu Vorhersagefehlern weit über der Baseline ungetunter linearer Modelle führen können. Im Gegensatz dazu reagieren regularisierte Modelle robust auf Parameteränderungen, stagnieren jedoch mit zunehmendem Horizont auf einem signifikant höheren Fehlerniveau, da sie die nichtlineare Tagesmuster nicht ausreichend abbilden können (Underfitting).

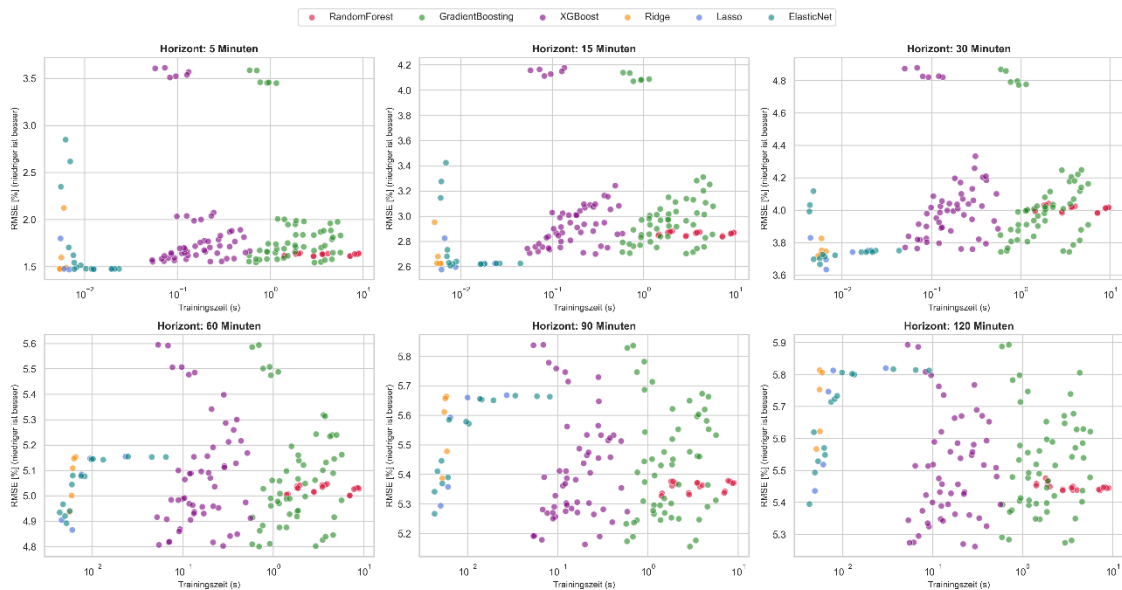


Abbildung 11: Effizienz der Detailed-Modelle aller Horizonte

Des Weiteren lässt sich aus Abbildung 11 ein deutlicher Unterschied in der Ressourceneffizienz linearer und baumbasierter Modelle ableiten. Die logarithmisch skalierte X-Achse stellt die mittlere Trainingszeit pro Kreuzvalidierungs-Split (Fold) für eine spezifische Hyperparameter-Kombination in Sekunden dar.

Lineare Modelle sind deutlich recheneffizienter und deren Vorhersagegüte schneidet im Vergleich mit baumbasierten Modellen bei kurzen Vorhersagehorizonten sehr gut ab. Mit zunehmendem Horizont setzen sich baumbasierte Modelle allerdings durch ein deutlich geringeres Fehlerniveau ab. Dieser Präzisionsgewinn kommt nur mit einem deutlich höheren Ressourcenaufwand im Modelltraining zustande, kommt allerdings durch die moderate absolute Trainingszeit von ca. 8 Minuten nicht zum Tragen. Dieser Mehraufwand ist durch das geringere Fehlerniveau gerechtfertigt und betrifft lediglich das Modelltraining und nicht die Inferenzzeit im produktiven Betrieb.

3.5 Inferenz und Frontend

Ist das Modelltraining abgeschlossen, werden die Modelle und die zugehörigen Skalierer in Pythons Pickle-Format serialisiert. Alle gelernten Parameter, Gewichte und Baumstrukturen werden in einen linearen Bytestrom transformiert

und auf den Datenträger geschrieben. Hierbei wird das Modell mit der höchsten Modellgüte gesondert markiert. Das ermöglicht es dem asynchronen HTTP-Server der Inferenz-Pipeline, die besten Modelle zu deserialisieren und schnell in den Arbeitsspeicher zu laden. Der HTTP-Server nimmt unbereinigte Live-Daten im Format der Rohdaten entgegen, bereinigt diese in einem identischen ETL-Prozess und orchestriert alle Modelle beider Modellfamilien (Quick & Detailed).

Das grafische Dashboard der Demo in Abbildung 12 visualisiert die Daten in einem übersichtlichen Graphen und liefert Funktionen, die eine Bedienung der Multi-Horizon-Modelle vereinfacht.



Abbildung 12: Dashboard der Inferenz-Demo (Detailed-Modus)

Im Dashboard erkennt man am Beispiel des Detailed-Modus einen blauen sowie einen orangen Polygonzug. Der blaue Polygonzug bildet die Auslastung am 1. Juni 2026 (Montag) von 17:10 bis 20:10 Uhr ab, während der orangefarbene Polygonzug die punktuellen Auslastungsprognosen der Detailed-Modelle bis zu zwei Stunden in der Zukunft visualisiert. Zudem gibt eine türkisfarbene Kurve zuzüglich eines Konfidenzintervalls im Hintergrund Auskunft darüber, wie sich die Auslastung bei gegebener Uhrzeit und Wetterlage für gewöhnlich verhält. Orangefarbene Markierungen auf der X-

Achse signalisieren dem Nutzer, ob Werte zu diesem Zeitpunkt im Rahmen des ETL-Prozesses imputiert wurden.



Abbildung 13: Dashboard der Inferenz-Demo (Quick-Modus)

Am Beispiel desselben Montags im Quick-Modus, sieht man in Abbildung 13, dass die Zeitleiste dynamisch gestaucht wird, um den Zeitraum vom Ausgangspunkt bis 24:00 Uhr abzubilden. Das ist möglich, da die durchgezogene türkisfarbene Prognosekurve des Quick-Modells nicht auf den maximalen Prognosehorizont der Detailed-Modelle (2 Stunden) beschränkt ist. Dafür ist die Prognose allerdings auch sichtbar ungenauer. Beide Modi erlauben das horizontale Ziehen des Graphen, um den Ausgangspunkt zu ändern, und blenden optional die Prognosekurven aller oder einzelner Modelle für den manuellen Vergleich ein. Für die Demo wurde der Ausgangspunkt in einen Zeitraum gelegt, dessen blaufarbener Auslastungspolygonzug bereits bekannt war, um die Modellgüte vorzuführen. Standardmäßig ist der Ausgangspunkt allerdings der letzte verfügbare Datenpunkt der eingelesenen Live-Daten.

4 Ergebnisse

4.1 Metriken der Modellgüte

4.1.1 Modellgüte der Quick-Modelle

Dafür, dass die Modellfamilie der Quick-Modelle ohne aussagekräftige Lag-Features auskommen muss, ist, wie in Tabelle 2 ersichtlich, das Random Forest Modell mit einem Bestimmtheitsmaß von 0,6555 bei einem RMSE von 4,28 für langfristige Auslastungsprognosen verwendbar. Es kann ca. 65 Prozent der Varianz der Auslastung erklären.

Modellname	R^2 -Score	RMSE (%)	Beste Hyperparameter
Random Forest	0,6555	4,28	max_depth: 10 min_samples_split: 5 n_estimators: 100
Gradient Boosting	0,6244	4,47	max_depth: 5 subsample: 1.0 n_estimators: 200 learning_rate: 0.01
XGBoost	0,6237	4,47	max_depth: 5 subsample: 0.8 n_estimators: 100 learning_rate: 0.05
ElasticNet	0,4778	5,27	alpha: 0.1 l1_ratio: 0.8
Ridge	0,4456	5,43	alpha: 100.0
Lasso	0,4205	5,55	alpha: 1.0
GBQuantile90	0,2116	6,47	max_depth: 4 n_estimators: 200 learning_rate: 0.1

Tabelle 2: Metriken der Quick-Modelle

4.1.2 Relevanz des Quantile Gradient Boosting Regressors

Die Metriken des Quantile Gradient Boosting Regressor sind ebenfalls in Tabelle 2 aufgeführt. Das Bestimmtheitsmaß von 0,2116 ist allerdings nicht mit den Ergebnissen der übrigen Modelle zu vergleichen, da diese dafür

Ergebnisse

prädestiniert sind, den Erwartungswert der Auslastung zu prognostizieren, indem sie den MSE minimieren. Die Quantilregression optimiert hingegen eine spezifische Verlustfunktion (Pinball-Loss). Das Ziel des GBQuantile90-Modells ist es nicht, den Mittelwert zu treffen, sondern bestimmte Verteilungsgrenzen (wie das 90%-Quantil) zu schätzen. Schätzt das Modell zu niedrig, wird der Vorhersagefehler mit 0,9 multipliziert und dadurch das Modell neunmal stärker bestraft als bei einer Überschätzung (Faktor 0,1). Das Modell erlernt diese Gewichtung und schiebt die Vorhersagelinie so lange nach oben, bis 90 Prozent aller echten Datenpunkte unter, und nur 10 Prozent über der Vorhersagelinie liegen. Das Modell ist darauf ausgelegt, Extremwerte an den Rändern der Wahrscheinlichkeitsverteilung abzubilden. So erklären sich der hohe RMSE und das niedrige Bestimmtheitsmaß. Daher sind die Metriken des GBQuantile90-Modells nicht mit den Metriken klassischer Regressionsmodelle vergleichbar und nicht in den XAI-Graphen dieser Arbeit enthalten, da sie diese verzerren würden.



Abbildung 14: Quantile Gradient Boosting Regressor Demo

Trotz der scheinbar geringeren Modellgüte (bewertet nach R^2) des Quantile Gradient Boosting Regressor Modells ist dessen Prognosekurve entscheidend für den produktiven Betrieb. Es prognostiziert, wie in Abbildung 14 zu erkennen, eine Auslastungsobergrenze, die mit 90-prozentiger Wahrscheinlichkeit nicht überschritten wird. Frühzeitige Informationen über potenzielle

Ergebnisse

Auslastungsspitzen bilden eine Entscheidungsgrundlage für z. B. die Ressourcen- und Wartungsplanung oder präventive Gebäudesteuerung (Klimatisierung) zu Stoßzeiten.

4.1.3 Modellgüte der Detailed-Modelle

Modellname	R^2 -Score	RMSE (%)	Beste Hyperparameter
Gradient Boosting (5 Minuten)	0,9617	1,43	max_depth: 3 subsample: 0.8 n_estimators: 500 learning_rate: 0.01
XGBoost (15 Minuten)	0,8920	2,40	max_depth: 3 subsample: 0.8 n_estimators: 500 learning_rate: 0.01
XGBoost (30 Minuten)	0,8043	3,22	max_depth: 3 subsample: 0.8 n_estimators: 500 learning_rate: 0.01
XGBoost (60 Minuten)	0,7082	3,94	max_depth: 3 subsample: 0.8 n_estimators: 500 learning_rate: 0.01
XGBoost (90 Minuten)	0,6895	4,06	max_depth: 3 subsample: 1 n_estimators: 500 learning_rate: 0.01
Gradient Boosting (120 Minuten)	0,6830	4,10	max_depth: 3 subsample: 0.8 n_estimators: 500 learning_rate: 0.01

Tabelle 3: Metriken der jeweils besten Detailed-Modelle pro Horizont

Die baumbasierten Ensemble-Modelle (XGBoost und Gradient Boosting) der Detailed-Modellfamilie schneiden durch aussagekräftige Lag-Features und die Trägheit der Auslastung über alle Horizonte hinweg sehr gut ab. Tabelle 3 fasst die Metriken der jeweils besten Modelle pro Horizont zusammen. Anhand der

Ergebnisse

sukzessiv abnehmenden Bestimmtheitsmaße kann man den natürlichen Zerfall der Vorhersagegüte beobachten. Die Modelle der 5- und 15-Minuten-Horizonte machen sich die hohe kurzfristige Trägheit der Auslastung (erkennbar in Abbildung 4 der Autokorrelationsanalyse) zunutze und erzielen exzellente R^2 -Scores von 0,96 und 0,89. Doch auch die Modellgüte der übrigen Detailed-Modelle ist der Modellgüte des besten Quick-Modells überlegen. Das belegt den Mehrwert der Zwei-Modellfamilien-Architektur, die saubere Implementierung der komplexen Multi-Horizon-Modellierung sowie die Relevanz der Konstruktion autoregressiver Features im Feature-Engineering.

4.2 Feature Decay

Neben der Evaluierung reiner Modellgüte und der praktischen Eignung offenbart die Analyse der Regressionsgewichte und Merkmalswichtigkeit beider Modellfamilien den Einfluss einzelner Merkmale und Effekte des Merkmalzerfalls über die verschiedenen Horizonte hinweg.

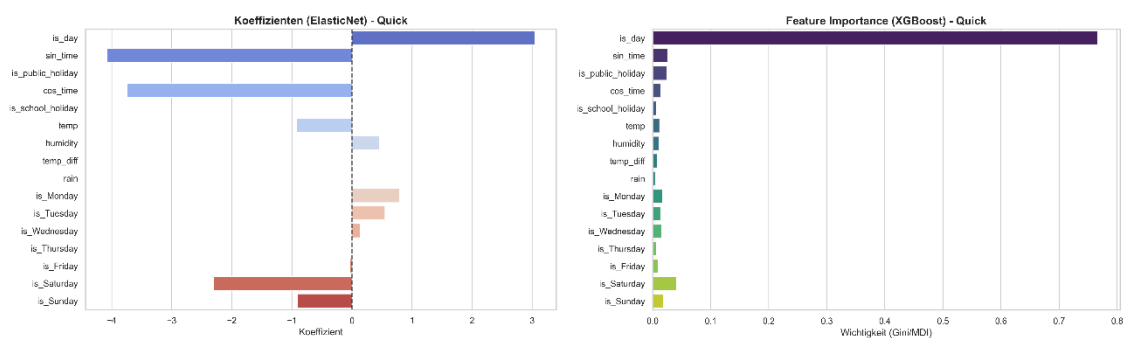


Abbildung 15: Koeffizienten und Merkmalswichtigkeit der Quick-Modelle

Im linken Graphen der Abbildung 15 sieht man die tatsächliche Wirkungsrichtung statistischer Merkmale, während der rechte Graph den Einfluss der Merkmale auf die Entscheidungsfindung des Quick-Modells visualisiert. Der rechte Graph legt nahe, dass der effektivste erste Schritt zur Auslastungsvorhersage die Beantwortung der Frage ist, ob Tageslicht vorhanden ist oder nicht. Das ElasticNet-Modell spricht diesem Merkmal im linken Graphen durch einen stark positiven Koeffizienten ebenfalls eine signifikant starke Bedeutung zu. Anschließend nutzt das XGBoost-Modell Informationen über Wochenenden, Feiertage sowie die aktuelle Uhrzeit im

Ergebnisse

Entscheidungsbaum, um das restliche Tagesprofil zu formen. Das ElasticNet gewichtet Informationen über Wochenenden und Uhrzeit ebenfalls stark, aber offenbart eine entscheidende Limitation linearer Modelle. Während XGBoost die Informationen über Feiertage aufgreift, regularisiert das lineare Modell den Koeffizienten auf 0, da es nichtlineare Effekte nicht abbilden kann. Dadurch sind lineare Modelle bei der Vorhersage nichtlinearer Tageskurven (ohne Lags) den baumbasierten Ensemble-Methoden systematisch unterlegen.

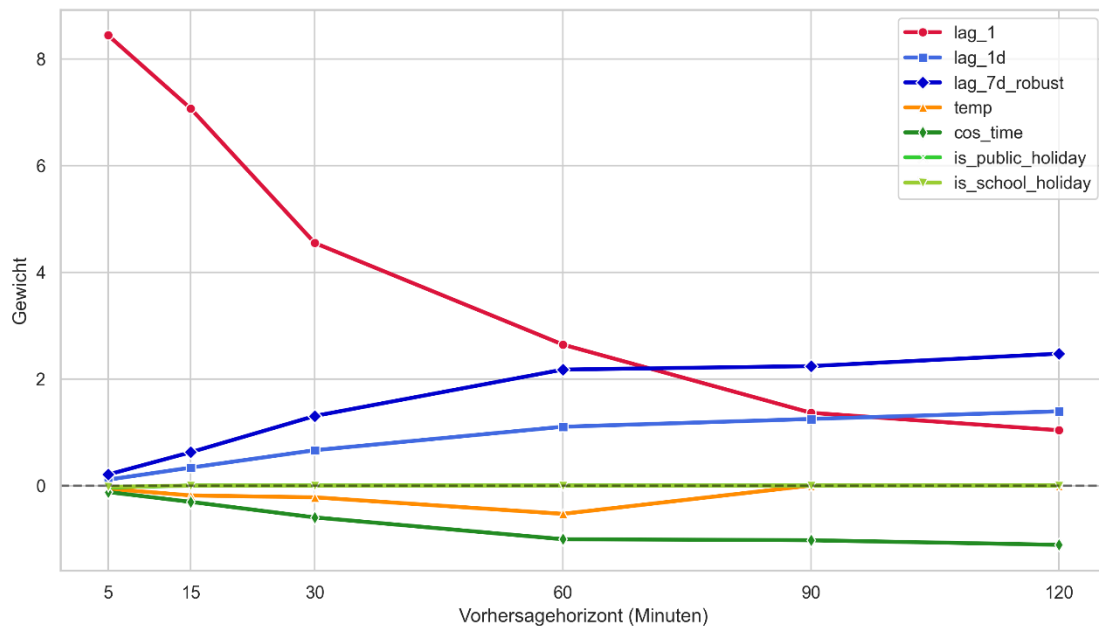


Abbildung 16: Koeffizientenverlauf über Zeithorizonte (ElasticNet)

Die XAI-Graphen der Detailed-Modelle visualisieren ebenfalls aufschlussreiche Dynamiken. Abbildung 16 zeigt erneut, wie das ElasticNet-Modell nichtlineare Effekte der Feiertage und Schulferien über alle Horizonte hinweg auf 0 regularisiert und für die Auslastungsprognose vollständig ignoriert. Außerdem ist der rapide Zerfall der Vorhersagekraft der kurzfristigen Lags anhand des roten Polygonzugs zu erkennen. Während kurzfristige Lags bei kurzen Vorhersagehorizonten dominieren, kompensiert das Modell den Informationsverlust bei zunehmendem Vorhersagehorizont, indem die Regressionsgewichte der historischen Saisonalitäten („lag_1d“ und „lag_7d_robust“) ansteigen. Zudem zeigt der dunkelgrüne Polygonzug, wie das Modell mit zunehmendem Vorhersagehorizont auf das „cos_time“-Merkmal zurückfällt, indem es einen

Ergebnisse

negativen Koeffizienten ansetzt. Da die Kosinuswelle der Uhrzeit nachts ihren Höhepunkt (+1) und tagsüber ihren Tiefpunkt (-1) erreicht, spiegelt das Modell die Welle über den negativen Koeffizienten. ElasticNet nutzt diesen negativen Faktor, um die natürliche Tagesamplitude zu rekonstruieren, sobald der starke Einfluss der kurzfristigen Lags mit zunehmendem Vorhersagehorizont nachlässt.

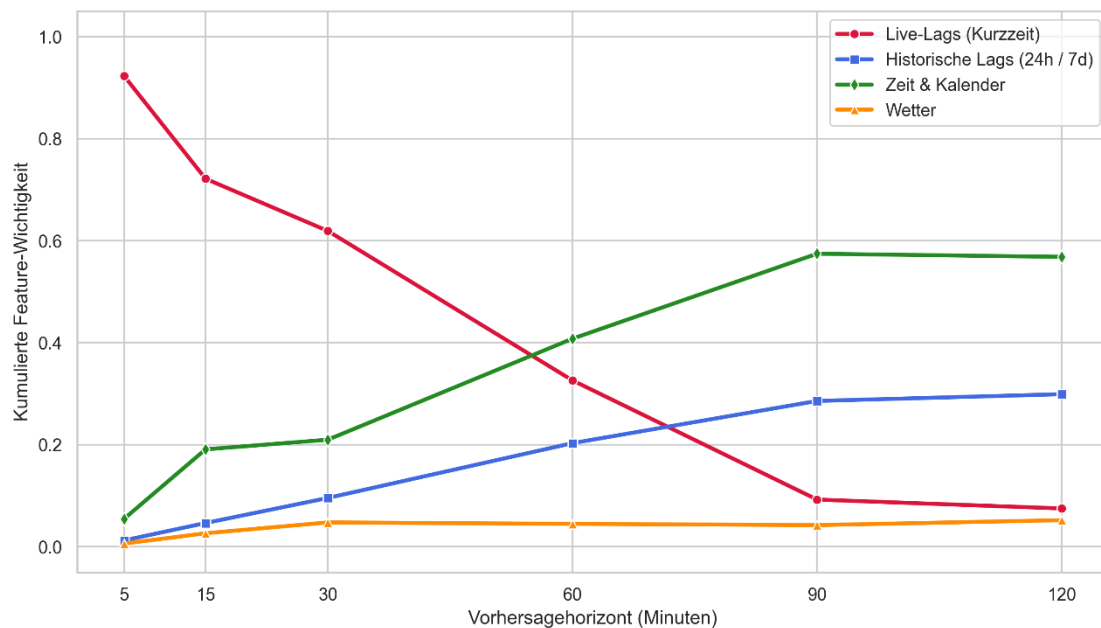


Abbildung 17: Bedeutungswandel der Feature-Gruppen (XGBoost)

Abbildung 17 zeigt, dass XGBoost nahezu identisch auf den Informationsverlust bei zunehmenden Vorhersagehorizonten reagiert. Im Gegensatz zum linearen Modell werden die nichtlinearen Feiertags- und Schulferieneffekte jedoch nicht ignoriert und fließen in die Entscheidungsfindung ein.

5 Kritische Reflexion und Ausblick

5.1 Zusammenfassung und Reflexion

Die durchweg stabile und hohe Modellgüte der Regressionsmodelle als Produkt der implementierten modularen End-to-End-Pipeline und Zwei-Modell-Architektur belegen eine saubere Methodik und beantworten die Forschungsfrage. Die Varianz der Fitnessstudioauslastung lässt sich vor allem mit den Modellen der Detailed-Modellfamilie zu 68 bis 96 Prozent, je nach Horizont, erklären. Über einen Horizont von 2 Stunden hinaus erklärt das beste Quick-Modell die Varianz zu 66 Prozent.

Dennoch unterliegt das System konzeptionellen und datenbedingten Limitierungen. Eine dieser Limitierungen ist die Datentiefe des historischen Datensatzes, die sich auf knapp zwei Monate des Frühlings beschränkt. Die kurze Beobachtungsdauer in diesem Zeitraum schließt die Erfassung meteorologischer Extremwerte wie Hochsommerhitze und Wintereinbrüche oder langfristigen menschlichen Verhaltens aus, was die Generalisierungsfähigkeit der Modelle auf das gesamte Kalenderjahr einschränkt. Zudem verhindert das gleitende Fenster von 365 Tagen vor der Modellierung, um Concept Drift zu vermeiden, die Analyse mehrjähriger Trends.

Außerdem schränken die Datenquelle sowie die Methodik der Datengewinnung die Modellierung ein. Die verwendete API des Fitnessstudios liefert lediglich den aggregierten Prozentwert der Auslastung, der in einem 5-Minuten-Intervall vom System abgerufen wird. Eine höhere Auflösung sowie wertvolle Prädiktoren wie z. B. die demografische Zusammensetzung der Besucher oder die Parkplatzbelegung vor Ort könnten die Vorhersagegüte der Modelle weiter präzisieren.

Zwei weitere methodische Schwächen liegen in der Vorhersagearchitektur bei der zeitlichen Auflösung sowie der Prognose von Auslastungsspitzen. Da die Modelle über den Mean Squared Error (MSE) optimiert wurden, zielen sie darauf ab, den statistischen Durchschnitt zu treffen. Das führt dazu, dass steile

Anstiege und plötzliche Auslastungsspitzen wie in Abbildung 14 unterschätzt werden. Zudem generieren die Detailed-Modelle keine kontinuierlichen Prognosekurven, da sechs starre Modelle trainiert wurden. Sie treffen lediglich sechs punktuelle Vorhersagen, die in der Demo interpoliert werden, wodurch das tatsächliche kontinuierliche Verhalten zwischen den Horizonten unbekannt bleibt.

5.2 Ausblick

Aus diesen Erkenntnissen ergeben sich methodische Weiterentwicklungspotenziale. Um plötzliche Auslastungsanstiege besser abzubilden, könnten abseits der MSE-Verlustfunktion alternative asymmetrische Verlustfunktionen, die eine Unterschätzung von Auslastungsspitzen stärker bestrafen, zu präziseren Vorhersagen führen. Alternativ könnte ein stärkerer Fokus auf Quantil-Regressionsmodelle gelegt werden, um Betreibern und Kunden obere Auslastungsgrenzen anstelle reiner Mittelwerte für die kurz- und langfristige Tagesplanung bereitzustellen.

Die Limitation der starren Vorhersagepunkte der Detailed-Modelle könnte durch einen Architekturwechsel von Einzelmodellen zu Multi-Output-Verfahren gelöst werden. Anstatt für jeden Horizont ein separates Modell zu trainieren, sollten Algorithmen entwickelt werden, die einen Zeitraum vorhersagen. Diese Modelle könnten eine kontinuierliche Liste an Vorhersagewerten in minütlicher Auflösung generieren, ohne, dass für jede Minute ein separates Modell trainiert werden muss. Hierbei bietet sich der Einsatz von Deep Learning an. Rekurrente neuronale Netze sowie Zeitreihen-Transformer sind im Gegensatz zu klassischen Regressionsverfahren speziell für die Verarbeitung sequenzieller Daten konzipiert und könnten hochauflösende, kontinuierliche Auslastungskurven generieren.

Abschließend kann die Generalisierungsfähigkeit der Modelle durch eine Erweiterung der Datengrundlage verbessert werden. Eine kontinuierliche Erfassung von mindestens einem gesamten Kalenderjahr ist erforderlich, um Wetter- und Saisonalitätseffekte zu validieren.

Literatur

- Brockwell, P. J., & Davis, R. A. (2016). *Introduction to Time Series and Forecasting*. Springer Cham. doi:<https://doi.org/10.1007/978-3-319-29854-2>
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning*. Springer Cham. doi:<https://doi.org/10.1007/978-3-031-38747-0>
- Jui-Sheng, C., & Duc-Son, T. (2018, Dezember 15). Forecasting energy consumption time series using machine learning techniques based on usage patterns of residential householders. *Energy*, 165, 709-726. doi:<https://doi.org/10.1016/j.energy.2018.09.144>
- Kapoor, S., & Narayanan, A. (2023, August 4). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*. doi:<https://doi.org/10.1016/j.patter.2023.100804>

Anhang

Anhang 1

Die folgenden Dateien sind dieser Arbeit separat als elektronischer Anhang beigelegt:

Dateiname	Beschreibung
source.zip	Vollständiger Quellcode der implementierten End-to-End-Pipeline (inkl. ETL, Modeltraining und Inferenz-Demo).
allmetrics.xlsx	Vollständige und tabellarisch aufbereitete Rohdaten der Evaluationsmetriken für alle 49 trainierten Modell.

Anhang 2

Ich erkläre, dass Ich

- ☒ mich aktiv über die Leistungsfähigkeit und Beschränkungen der in meiner Arbeit eingesetzten KI-Systeme informiert habe;
- ☒ alle Inhalte aus wissenschaftlich anerkannten Quellen entnommen und entsprechend gekennzeichnet habe; alle Inhalte unter Anwendung wissenschaftlicher Methoden im Rahmen der vorliegenden Arbeit von mir selbst entwickelt wurden;
- ☒ mir bewusst ist, dass ich als Autor dieser Arbeit die Verantwortung für die in meinen gemachten Angaben und Aussagen trage.

Bei der Erstellung der Arbeit habe ich die folgenden auf künstlicher Intelligenz (KI) basierten Systeme in der im Folgenden dargestellten Weise benutzt:

Arbeitsschritt	Eingesetzte(s) KI-System(e)	Beschreibung der Verwendungsweise
Generierung von Ideen und Konzeption der Arbeit	Gemini 3.1 Pro	Initiales Mockup der Gliederung auf Basis der existierenden Projektdokumentation.
Literatursuche	Gemini 3.1 Pro	Deep Research mit der Vorgabe, moderne, seriöse link.springer.com Quellen zu bevorzugen.

Arbeitsschritt	Eingesetzte(s) KI-System(e)	Beschreibung der Verwendungsweise
Literaturanalyse	Gemini 3.1 Pro	„Fuzzy Keyword Search“
Literaturverwaltung und Zitationsmanagement	n/a	n/a
Auswahl von Methoden und Modellen	Gemini 3.1 Pro	Generierung von Vorschlägen zum methodischen Vorgehen in der Implementierung auf Basis wissenschaftlicher Standards.
Datensammlung und Analyse	Gemini 3.1 Pro Code2Prompt	Generierung einer detaillierten Zusammenfassung der Codebase.
Generierung von Programmcodes	Gemini 3.1 Pro Gemini 3.5 Flash	Generierung grober Architektur und Boilerplate anhand konkreter Anforderungen.
Erstellung von Visualisierungen	Gemini 3.1 Pro Gemini 3.5 Flash	Erstellung von Matplotlib-Graphen anhand konkreter Anforderungen.
Interpretation und Validierung	n/a	n/a
Strukturierung des Texts der Arbeit	n/a	n/a
Formulierung des Texts der Arbeit	Gemini 3.1 Pro	Umformulierung bestehender Textentwürfe.
Übersetzung des Texts der Arbeit	n/a	n/a
Redigieren des Texts	LanguageTool Pro	Im „Picky-Mode“ mit wissenschaftlichem Fokus.
Vorbereitung der Präsentation des Texts	n/a	n/a

■■■■■■■■■■, 12. Juli 2026

■■■■■■■■■■

■■■■■■■■■■